Names:
Logan Hunter, u1342460
Quang Nguyen, u1470229
Robbie DeMars, u1411323

Introduction & Background:

This project aims to tackle the problem of predicting an athlete's performance in the Olympics using machine learning. We want to be able to take in an athlete's features like height, weight and age and use it to predict their performance using a classification model. The current alternative is to scout athletes individually which is time intensive and introduces bias from the scout. By using more concrete data like results and physical measurements we're hoping to have an accurate model to help predict a future athlete's performance in certain events without any bias. Our approach involves aggregating multiple datasets with a mix of specialized and general data. We're hoping with this approach we can get a more thorough final dataset to help increase the accuracy of our model.

To tackle this approach our final dataset should have a variety of different factors that could help predict the final prediction. Furthermore, since we're training a classification model the final dataset should be as comprehensive and complete as possible. A major challenge in creating a comprehensive dataset is that olympics data is often fragmented between sports or lacks specific information we want our model to train on. The fragmented and spare nature of the datasets means that joining these tables in a naive way could lead to noisy results due to differing schemas and format. This could include missing data, duplicate data and inconsistent data.

To address this problem our final approach starts by cleaning up individual datasets based on a pre-defined schema. This process involved standardizing times, medals and athlete names to help aid the final join step. By forcing the data into a pre-defined schema there will often be missing values since not all datasets will have the same information. However, by merging several datasets together it should help aid in filling the gaps since we have a mix of general and specific data.

Data Used:

Datasets:
Scraped wikipedia data

- Wikipedia

GDP Dataset

- Maddison Project

Small dataset

- Swim
- Track
- General

For our datasets, we initially decided on three Kaggle datasets, one containing lots of information on swim candidates, one for track candidates, and one that has lots of height / weight data and other general information. We also got a dataset from the Maddison Project which is the most expansive dataset on GDP estimations we could find for countries globally. All of these datasets together covered the schema we were planning for. Data on swimmers, runners, physical attributes like height and weight, and a GDP table to tie to important years. Unfortunately we realized, together they all had a lot of spaces in terms of filled in information, so we were going to have to look for an alternative source to fill in this data. We came up with the solution with the professor's permission to make Claude scrape Wikipedia for all the information it can find regarding the relevant values in our schema. This seemed to be a decent solution to this problem.

Major shortcomings on our data include that as you go farther back in time, details about olympics competitors becomes more sparse, as well as our data still missing a decent amount of data points that we just couldn't find. Some features such as names were broken enough to be unrepairable too. Unfortunately, the Wikipedia data was especially difficult to repair, as it was retrieved from hundreds of articles written by different people, so data / pages were often represented differently. Also simple features about a person such as height and weight aren’t the most explanatory, so it would be nice to get more real data on each person such as wingspan, something suspected to help swimmers, as well as other more personal data on each person.

Motivation

Olympic scouting is a traditionally subjective process, often relying on intuition and limited data. Coaches and teams can evaluate potential candidates based on different statistics like race time, their physical builds, their national economic context, but it is difficult to find a clean dataset that combines these attributes together. Our project is motivated by the idea that a data-centric machine learning pipeline can reduce subjectivity and bias, providing objective, consistent insights about an athlete’s medal potential.

Rather than trying to create a new classification algorithm, we wanted to construct a robust canonical dataset from multiple messy sources. This approach aligns perfectly with data-centric AI, which utilizes the idea that improving data quality often results in better gains than tweaking model architectures. By resolving duplicates, normalizing names, and integrating economic indicators like GDP-per-capita, our dataset captures many of the factors that influence Olympic success.

Design

Data reading and normalization

We load four original entity groups (events, athletes, athlete-event results, Olympics) from multiple CSV files each. All of the tables are unioned with fallback columns, text is normalized by trimming, lowercasing, stripping HTML entities and other garbage characters. Country codes are unified to IOC three letter codes with a GDP derived lookup.

Event Resolution

Event names are parsed with a custom regex UDF that extracts distance, gender, discipline, and round. A tokenization step builds word-level tokens for Jaccard similarity. Candidate pairs that share the same Olympics ID, event type, and parsed distance are assigned a weighted score based on their matches for each category (distance, gender, token Jaccard, discipline). Pairs above a threshold (0.825) become edges in a graph. GraphFrames' connectedComponents assigns each row a raw event cluster ID.

Olympic deduplication

Host city names have variations ("Rio" vs "Rio de Janeiro") are handled by blocking on year, season, and the first five characters of the normalized city, then computing Levenshtein similarity. High scoring pairs are clustered, and later we collapse by year and season for one canonical Olympic ID per real-world edition.

Athlete name cleaning

Our athlete resolution pipeline contains a rigorous amount of name-cleaning that includes:

- Unicode NFC normalization to properly handle accents and diacritics,
- Removal of smart quotes and garbage parentheses,
- Filtering to Unicode Letters, hyphens, and apostrophes,
- A minimum length requirement of 2 characters,
- Suffix detection and swapping (e.g., "jr" becomes middle name),

- A global first name correction map replaces short, rare names (length <= 4, frequency > 30) with the most frequent longer name that only adds characters. (Levenshtein distance <= 2 and equals the length difference). This repairs source file name corruptions like pter → peter before clustering.
- A separate hyper conservative last name correction is applied only to last names that appear in a single country, have frequency < 30 and have much more common longer variant that shares the first two letters.
- Removal of obvious event-description phrases mistaken as names.

Relay explosion and blocking

Rows that concatenate multiple athletes are detected and split into individual records using a custom UDF that handles first and last name boundaries. After cleaning, we generate a strict block key (first initial + last name prefix + birth decade + gender + country) as well as a relaxed block key for athletes missing birth year. A third ultra restrictive block joins rows on exact last name, gender, and country, keeping pairs whose first names differ by a Levenshtein distance of 2 or less.

Similarity scoring and clustering

For every candidate pair we compute exact first/last match, birth-year match, Jaccard token similarity, and full-name Levenshtein similarity. The four scores are combined with fixed weights. Strict and relaxed key pairs must be >= 0.825. The Levenshtein-block pairs use a lower threshold (0.55) because the block itself is already extremely strict. All surviving pairs become edges in a graph. GraphFrames' connected components produce a raw athlete cluster.

Canonical tables

In each cluster we pick the longest first-name variant as the canonical candidate as a final guard against truncation, then aggregate other attributes (height, weight, country, birth-year). A final identity-based dedup (first, last, nationality, gender) removes any remaining duplicates. Canonical and Olympics are built manually. A final integrity check removes rows where athlete gender mismatches event gender, and discards participation where the athlete would have been older than 80 or younger than 10 at the games.

The pipeline outputs two sets of CSVs: **clean_datasets/** which matches our schema, and **canonical/** which is the aggregated version.

Outperforms simple joins

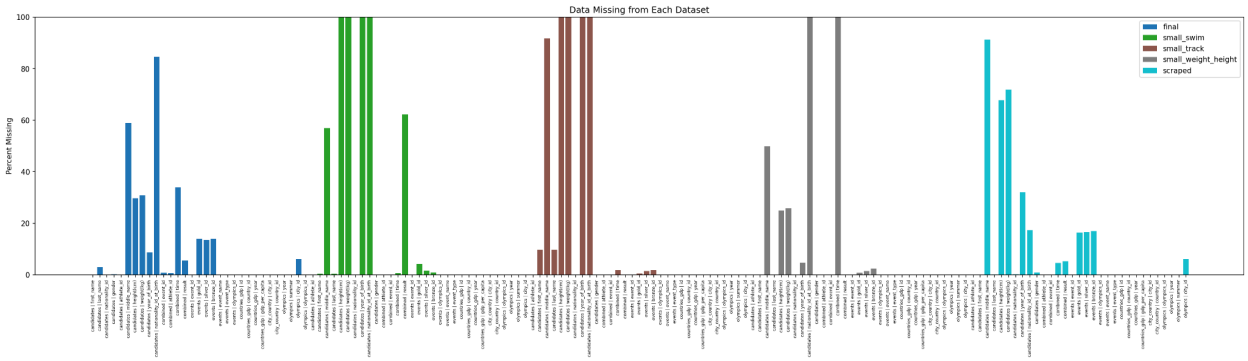
Existing Olympic dataset mergers rely on exact name or ID matches, which fail when IDs aren't universal, names are misspelled, or country codes are inconsistent. Our

approach uses fuzzy similarity, multi-field blocking, and graph based clustering to link records without a shared key, making it robust to the noise and missing data that are common in scraped sources.

Our design is modular, meaning each section (events, Olympics, athletes) can be inspected independently, and new data sources can be added by simply loading and unioning them before clustering.

Evaluation:

To evaluate our dataset, we first wanted to compare how complete each one is independently, to show that we had successfully increased overall data coverage with our methods. We did this by counting the number of missing values in each column of each table of each dataset, that is a mouthful. What that means in the table below is that each color represents a different dataset, and each column represents a corresponding percent missing of each column, in each table, in said dataset.



[Link to High Resolution Image]

Next what we wanted to do was to analyze more generally, the worst missing percentage in the entirety of each dataset, visually, this was done by taking the longest bar in each color category above. We also wanted to analyze the average percent of missing values, as the worst value does not necessarily show improvement across columns as holistically. You can see these results in the table below.

Dataset	Worst Percent Missing	Average Percent Missing
final	84.7	9.8
scraped	91.4	11.2
small_swim	100	17
small_track	100	16.7

small_weight_height	100	10

Finally, we wanted to analyze our data via a classification model. While we learned a lot in class about a Random Forest model, looking into it, people actually preferred a different model for tasks such as this, XGBoost (Extreme Gradient Boost). Where Random Forest is a bunch of bootstrapped models, this model instead uses multiple models to update each successive one's gradients. While we chose based on testing each combination, this is likely better because it is more responsive to noisy data than Random Forest's balancing many models approach.

Next we needed to tackle our imputation method. We assumed we would need to do a K-Nearest Neighbors approach, but our tests actually found that a simple median imputer outperformed KNN in every case. This we theorize is because since there is a major class imbalance in our data, KNN actually makes way more medalists than there actually should be, which results in the underlying correlations getting polluted. The median of course in the big dataset has way more non medalists than medalists, so a median approach will replace all missing values with no medal, which seemed to be more true than the alternative.

Below you can see the results of running our imputation-model combination tests. We primarily focused on accuracy for deciding which combination is the best in this case, but it's important to note the low F1-Scores.

Accuracy: 71.4%		KNN + XGB		
	Precision	Recall	F1-Score	Support
No Medal	0.74	0.99	0.84	6281
Gold	0.48	0.18	0.26	967
Silver	0.27	0.03	0.06	900
Bronze	0.32	0.06	0.09	882
Accuracy	-	-	0.71	9030
Macro Avg	0.45	0.31	0.32	9030
Weighted Avg	0.62	0.71	0.63	9030

Accuracy: 72.2%		Median + XGB		
	Precision	Recall	F1-Score	Support
No Medal	0.76	0.98	0.86	6281
Gold	0.44	0.22	0.29	967
Silver	0.3	0.09	0.13	900
Bronze	0.3	0.08	0.12	882
Accuracy	-	-	0.72	9030
Macro Avg	0.45	0.34	0.34	9030
Weighted Avg	0.64	0.72	0.65	9030

Accuracy: 68.9%		Median + RF		
	Precision	Recall	F1-Score	Support
No Medal	0.79	0.88	0.83	6281
Gold	0.41	0.35	0.38	967
Silver	0.29	0.21	0.24	900
Bronze	0.33	0.23	0.27	882
Accuracy	-	-	0.69	9030
Macro Avg	0.46	0.41	0.43	9030
Weighted Avg	0.65	0.69	0.67	9030

Accuracy: 68.5%		KNN + RF		
	Precision	Recall	F1-Score	Support
No Medal	0.78	0.88	0.83	6281
Gold	0.41	0.33	0.37	967
Silver	0.29	0.2	0.24	900

Bronze	0.32	0.21	0.26	882
Accuracy	-	-	0.69	9030
Macro Avg	0.45	0.41	0.42	9030
Weighted Avg	0.65	0.69	0.66	9030

Noting the F1 Score on all of these is relatively low, we moved forward instead prioritizing bringing up the F1 Score for model parameter tuning. Optimizing for the F1-Score we felt was a better representation of our data's robustness, rather than a model potentially learning a cheat since the F1-Score is a combination of the Precision and Recall. This did come at the cost of the model's accuracy, but is a better representation of the data overall. Below you can see the model's performance with each of our datasets, Scraped, Smaller_Swim, Smaller_Track, Smaller_Width_Height, and Final.

Accuracy: 64.2%		Scraped		
	Precision	Recall	F1-Score	Support
No Medal	0.93	0.73	0.82	12332
Gold	0.34	0.44	0.39	1362
Silver	0.19	0.30	0.23	1395
Bronze	0.17	0.37	0.23	1265
Accuracy	-	-	0.64	16354
Macro Avg	0.41	0.46	0.42	16354
Weighted Avg	0.76	0.64	0.69	16354

Accuracy: 51.7%		Smaller_Swim		
	Precision	Recall	F1-Score	Support
No Medal	0.73	0.75	0.74	457
Gold	0.22	0.22	0.22	95
Silver	0.14	0.12	0.13	98

Bronze	0.08	0.09	0.09	92
Accuracy	-	-	0.52	742
Macro Avg	0.3	0.3	0.3	742
Weighted Avg	0.51	0.52	0.51	742

Accuracy: 26.6%		Smaller_Track		
	Precision	Recall	F1-Score	Support
Gold	0.26	0.24	0.25	152
Silver	0.28	0.29	0.28	148
Bronze	0.27	0.27	0.27	139
Accuracy	-	-	0.27	439
Macro Avg	0.27	0.27	0.27	439
Weighted Avg	0.27	0.27	0.27	439

Accuracy: 63%		Smaller_Weight_Height		
	Precision	Recall	F1-Score	Support
Gold	0.65	0.64	0.64	2190
Silver	0.61	0.61	0.61	2147
Bronze	0.63	0.65	0.64	2197
Accuracy	-	-	0.63	6534
Macro Avg	0.63	0.63	0.63	6534
Weighted Avg	0.63	0.63	0.63	6534

Accuracy: 69.2%	Final

	Precision	Recall	F1-Score	Support
No Medal	0.88	0.90	0.88	4423
Gold	0.51	0.53	0.52	1460
Silver	0.49	0.43	0.46	1450
Bronze	0.51	0.49	0.50	1461
Accuracy	-	-	0.69	8794
Macro Avg	0.59	0.59	0.59	8794
Weighted Avg	0.69	0.69	0.69	8794

As you can see, our Final data's Macro Avg F1-Score surpasses all alternatives in both accuracy and F1-Score! It is worth noting that while Smaller_Weight_Height has a higher F1-Score, the dataset is unable to classify whether a candidate is not a medalist, so it fails to achieve our purposes.

Conclusion:

This project shows that a data-centric pipeline that uses heavy cleaning, normalization, and entity resolution, can transform a collection of dirty, overlapping datasets into a reliable canonical database prepared for machine learning. The resulting dataset combines information from four distinct sources, combines them together for cleaning and data reparation, and creates tangible features for machine learning to be effective.

Strengths

We handled severe data-quality issues. Spelling variants, missing birth-years, concatenated relay teams, and corrupted diacritics (dots, squiggles, etc.). Entity resolution was performed without any universal ID, using custom fuzzy matching and GraphFrames clustering. The output of both schema-aligned clean tables and aggregated canonical tables gives us a readable analytical view. Integrating socioeconomic indicators with GDP-per-capita added a novel feature dimension for future models.

Weaknesses

Our global first-name correction map successfully repaired many source corruptions (e.g., pter abay → peter abay (real athlete)) by replacing short, rare first names with the most frequent longer name that differed only by missing characters. However, this

injection of synthetic corrections risks reassigning a very rare but legitimate short name to an incorrect longer variant. We mitigated this risk with name frequency thresholds and strict edit-distance equality checks. We did not run this name correction on last-names, because it carried too high a risk of corrupting data further. Our conservative last-name correction that only applied to country-isolated , extremely rare surnames, avoids touching globally common short names like “Li” or “Wu”, but again cannot guarantee zero bad corrections. Despite these safeguards, some names may be imperfect.

Another glaring “weakness” is our current implementation of the windows partitions that generate readable IDs. These partitions constantly have to go row by row to fulfill the sequential ID requirement we set. While we had an implementation version that simply used the raw cluster IDs and were non-sequential that ran faster, we decided that the readability was more important than the time it took to run the code.This was more of a design choice than a weakness.

Future

The event parser and clustering can be used for all Olympic sports rather than just Athletics and swimming, which would create a complete comprehensive dataset. Adding past performance features like an athlete’s previous race times or career progression would showcase the time-series dimension currently missing. The spark pipeline can be optimized by replacing single-partition window operations with parallel non-sequential ID generation, removing the memory bottleneck. The entire workflow could be collapsed into a single reusable function that reads a new CSV, maps it to the canonical schema, cleans the data, and outputs a model-ready file.